

Cyclomatic Complexity Analyzer for C Programs

Design and Implementation of a Lexer–Parser–CFG Pipeline with Web-Based Visualization

Siddharth Narayan Mishra
Roll Number: 123CS0189

Software Testing Laboratory

1 Aim

To design and implement a tool that computes the **cyclomatic complexity** of functions written in the C programming language. The tool parses a C source file, builds a **Control Flow Graph (CFG)** for every function it contains, applies McCabe’s cyclomatic complexity formula to each CFG, and presents the result through an interactive web interface that visualizes the graph alongside the source code.

2 Theory

2.1 Cyclomatic Complexity

Cyclomatic complexity, introduced by Thomas J. McCabe in 1976, is a static software metric used to measure the structural complexity of a program by counting the number of linearly independent paths through its control flow. It is widely used in software testing to:

- Estimate the minimum number of test cases required to achieve branch coverage,
- Flag overly complex functions that are more likely to contain defects and are harder to maintain,
- Guide basis-path testing, where a set of independent paths through the CFG is chosen as the test suite.

For a control flow graph G with a single entry and a single exit, McCabe’s formula is

$$M = E - N + 2P \tag{1}$$

where

- E is the number of edges in the graph,
- N is the number of nodes (basic blocks) in the graph,
- P is the number of connected components (for a single function, $P = 1$).

Equivalently, M can be computed as the number of decision points (predicate nodes such as **if**, **while**, **for**, **case**) plus one, since every binary decision adds exactly one edge more than it adds nodes.

2.2 Control Flow Graphs

A control flow graph represents a function as a directed graph in which:

- Each node is a **basic block** — a maximal straight-line sequence of statements with no jumps into or out of its middle,
- Each edge represents a possible transfer of control between blocks,
- A single **ENTRY** node and a single **EXIT** node bound the function.

Branching constructs (`if`, loops, `switch`) split a block into a predicate node with two or more outgoing edges (labelled `true/false`, or `case` values), while sequential statements are merged into the same block wherever no jump target lies between them.

3 Tools and Technologies

Component	Tool / Library
Lexical analysis	Flex (<code>lexer.1</code>)
Syntax analysis	GNU Bison (<code>parser.y</code>), LALR(1) grammar
AST & CFG construction	C (<code>ast.c</code> , <code>cfg.c</code>), compiled with GCC
Macro/preprocessor handling	<code>gcc -E -P</code> (piped input)
Backend API	Node.js, Express, Multer (file upload)
Frontend UI	React (Vite), <code>@xyflow/react</code> (React Flow)
Graph auto-layout	<code>dagre.js</code>

Table 1: Tools used across the pipeline

4 System Architecture

The system is organized as three independently runnable components that communicate over HTTP, and internally the analyzer itself is a compiler-style pipeline. Figure 1 shows the overall flow from a `.c` source file to the rendered CFG in the browser.

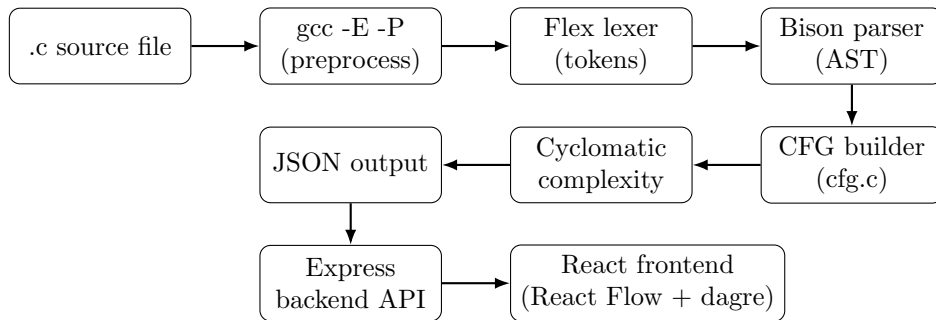


Figure 1: End-to-end pipeline: source $\rightarrow$ preprocessing $\rightarrow$ lexing $\rightarrow$ parsing $\rightarrow$ AST $\rightarrow$ CFG $\rightarrow$ complexity $\rightarrow$ JSON $\rightarrow$ backend API $\rightarrow$ frontend visualization.

1. **Parser (C, `parser/`):** a native binary, `cflow-analyzer`, built from a Flex lexer and a Bison grammar. It reads one `.c` file, builds an abstract syntax tree (AST), constructs one CFG per function, computes the cyclomatic complexity of each, and emits a JSON document describing every basic block and edge.

2. **Backend (Node.js/Express, backend/)**: a thin HTTP wrapper that invokes the compiled binary as a subprocess (`execFile`) on an uploaded or sample `.c` file and streams its JSON output back to the browser.
3. **Frontend (React, frontend/)**: renders the CFG returned by the backend as an interactive node-link diagram, shows the source file with click-to-locate highlighting between source lines and CFG nodes, and displays the $M = E - N + 2P$ computation for the selected function.

5 Implementation

5.1 Lexical Analysis

The lexer (`lexer.1`) tokenizes a preprocessed C source file. Since macro expansion and conditional compilation (`#ifdef`) are already resolved by `gcc -E` before the lexer ever sees the input, the lexer itself only needs to recognize:

- C keywords (`if`, `else`, `while`, `for`, `switch`, `case`, `return`, type keywords, etc.),
- identifiers, integer/float/hex/character constants, and string literals,
- multi-character operators (`++`, `->`, `<=`, `&&`, compound assignment operators, ...),
- single-character punctuation, forwarded verbatim as their own token.

Comments and residual `gcc -E` linemarkers (e.g. `# 1 "file.c"`) are discarded in the lexer itself.

5.2 Syntax Analysis and AST Construction

The grammar (`parser.y`) is a Bison LALR(1) grammar implementing a practical subset of C89-style expression and statement syntax: function definitions, all standard statement forms (`if/else`, `while`, `do...while`, `for`, `switch/case/default`, `break`, `continue`, `return`, `goto/labels`), declarations, and the full C expression-precedence chain. The precedence chain runs from `primary_expr` up through `assignment_expr`, and uses the classic `%prec LOWER_THAN_ELSE` technique to resolve the dangling-else ambiguity.

Rather than building a full C type system, each expression rule reconstructs and returns its own source text (e.g. `"a[i] < 0"`) which is later used verbatim as the label of the basic block it is attached to. Each statement is represented as an `Stmt` node (Listing 1) tagged with a `StmtType` and up to four child-statement pointers, whose meaning depends on the type of statement (e.g. `s1/s2` are the then/else branches of an `if`, while `s1` is the loop body of a `while`).

Listing 1: Core AST node definition (`ast.h`)

```

1 typedef enum {
2     STMT_COMPOUND, STMT_IF, STMT_WHILE, STMT_DO_WHILE, STMT_FOR,
3     STMT_SWITCH, STMT_CASE, STMT_DEFAULT, STMT_BREAK, STMT_CONTINUE,
4     STMT_RETURN, STMT_GOTO, STMT_LABEL, STMT_EXPR, STMT_DECL, STMT_EMPTY
5 } StmtType;
6
7 typedef struct Stmt {
8     StmtType type;
9     int line;
10    char *text;                /* expr / cond / label / goto target */
11    char *text2, *text3;       /* STMT_FOR only: init, incr */
12    struct Stmt *s1, *s2, *s3, *s4; /* meaning depends on type */
13    struct StmtList *body;      /* STMT_COMPOUND: statements in block */
14 } Stmt;

```

5.3 Control Flow Graph Construction

The CFG builder (`cfg.c`) walks the AST of each function once, threading a “current block” pointer through the statement list and emitting new basic blocks and edges as control-flow constructs are encountered. Table 2 summarizes the graph fragment generated for each construct.

Construct	CFG fragment emitted
Sequential statement	Appended as another line into the current block; no new node.
<code>if/else</code>	Condition ends the current block; true edge to a new then-block, false edge to a new else-block (or directly to the join block if no else); both branches merge into a new join block.
<code>while</code>	New header block holding the condition; true edge into the body, false edge to the exit block; the body’s fall-through edges back to the header.
<code>do...while</code>	Body executes unconditionally once; the trailing condition block has a true edge back to the body and a false edge to the exit.
<code>for</code>	Init text is appended to the entry block; a header block holds the condition (true →body, false →exit); the body’s exit unconditionally reaches an increment block, which loops back to the header.
<code>switch</code>	Each case/default becomes its own block reached by a case -labelled edge from the switch’s condition block; blocks without a preceding break fall through to the next case block, matching real C semantics.
<code>break</code> <code>continue</code>	/ Unconditional edge to the innermost enclosing loop’s exit block / loop-continuation block, tracked via explicit stacks (<code>break_stack</code> , <code>continue_stack</code>).
<code>return</code> <code>goto/label</code>	Unconditional edge straight to the function’s single EXIT node. Recorded as a pending jump and resolved in a second pass once every label in the function has been visited, since a goto may jump forward.

Table 2: Mapping from AST statement types to CFG fragments

Every function’s CFG has exactly one **ENTRY** block and one **EXIT** block, so $P = 1$ always holds, which is what allows the simplified formula $M = E - N + 2$ to be used directly (Listing 2).

Listing 2: Cyclomatic complexity computation (`cfg.c`)

```

1 int cyclomatic_complexity(CFG *cfg) {
2     /*  $M = E - N + 2P$ ,  $P = 1$  (single entry/exit per function) */
3     return cfg->num_edges - cfg->num_blocks + 2;
4 }

```

Once every CFG is built, `cfg_to_json()` serializes each function’s node list (id, source-line range, label text) and edge list (source, target, edge type, optional case label) to a JSON document that is written to stdout or a given output file.

5.4 Backend and Frontend

The Express backend (`server.js`) exposes two endpoints: `GET /api/sample`, which analyzes a bundled example file, and `POST /api/analyze`, which accepts an uploaded `.c` file (via Multer, capped at 1 MB), writes it to a temporary path, and invokes the compiled analyzer binary as a subprocess before returning its JSON output. Both endpoints respond with the original source

text alongside the per-function CFG data so the frontend can correlate graph nodes with source lines.

The React frontend renders the selected function’s CFG using `@xyflow/react`, with node positions computed automatically by `dagre`’s layered (Sugiyama-style) layout algorithm. Edges are colour-coded by type (true/false/unconditional/case), and clicking a node highlights the corresponding source-line range, while clicking a source line highlights the block that contains it. A metrics panel displays N , E , P , and the resulting M for the active function.

6 Results

The analyzer was run on two hand-written test programs, `sample1.c` and `sample2.c`, exercising `if/else if` chains, loops (`while`, `do...while`, `for`, nested `for`), `switch` with fall-through, and a `goto`. Table 3 shows the computed metrics for every function.

File	Function	Nodes (N)	Edges (E)	Complexity (M)
sample1.c	classify	8	9	3
sample1.c	sumEvens	8	9	3
sample1.c	gradeOf	8	11	5
sample1.c	findFirstNegative	7	8	3
sample2.c	binarySearch	11	13	4
sample2.c	validateAndSum	12	14	4
sample2.c	collatzSteps	8	9	3
sample2.c	categorize	10	15	7
sample2.c	gcd	5	5	2
sample2.c	matrixTrace	12	14	4

Table 3: Cyclomatic complexity of every function in the two test programs

These results match hand-verified expectations: `gcd` has a single loop and no other branching, giving $M = 2$ (one decision point plus one); `categorize` has the highest complexity because its six-way `switch` contributes one decision edge per `case` label.

6.1 Worked Example: `classify()`

Consider the following function from `sample1.c`:

```

1 int classify(int x) {
2     int result = 0;
3     if (x < 0) {
4         result = -1;
5     } else if (x == 0) {
6         result = 0;
7     } else {
8         result = 1;
9     }
10    return result;
11 }
```

The tool decomposes this into 8 basic blocks and 9 edges (Figure 2): an entry block holding the declaration and the first condition, two nested decision blocks (from the `if/else if` chain), three assignment blocks, a join block, and the return/exit blocks.

Applying the formula: $M = E - N + 2P = 9 - 8 + 2(1) = 3$, matching the two `if` decision points plus one, and agreeing exactly with the tool’s own output. This also indicates that three

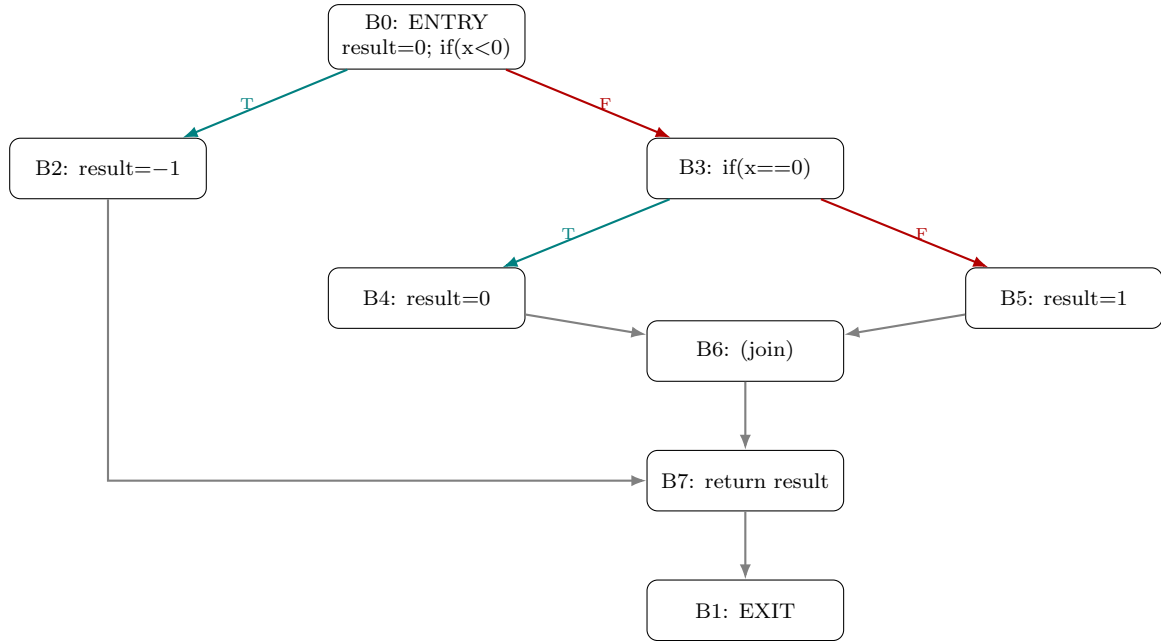


Figure 2: CFG generated for `classify()`: $N = 8$, $E = 9$, so $M = 9 - 8 + 2 = 3$.

independent test cases – $x < 0$, $x = 0$, and $x > 0$ – are sufficient to exercise every branch of the function, which is precisely the practical use of cyclomatic complexity in basis-path testing.

7 Known Limitations

The current implementation targets a practical subset of C sufficient for algorithmic function bodies, and does not attempt full ISO C conformance:

- No `typedef` support, and consequently no “lexer hack” to disambiguate typedef’d type names from identifiers.
- No explicit type-cast expressions, e.g. `(int)x`.
- `#included` headers are stripped before parsing rather than resolved – only the function bodies defined in the user’s own file are analyzed.
- Global variable declarations support only a single declarator per line.

8 Conclusion

This lab exercise implements a complete, working front-to-back pipeline for static analysis of C programs: a Flex/Bison compiler front end builds an AST from C source, a graph-construction pass turns each function’s AST into a control flow graph following well-defined per-construct rules, and McCabe’s cyclomatic complexity formula $M = E - N + 2P$ is applied directly to the resulting graph. Wrapping the analyzer in an HTTP API and a React-based visualizer made it possible to inspect and verify the generated CFGs interactively, which was invaluable for validating tricky constructs such as `switch` fall-through and forward `goto` resolution against their expected complexity values. The exercise reinforced both the theory of control-flow-based test-adequacy criteria and the practical mechanics of building a small compiler front end with Flex and Bison.